Operating Systems - Revision Notes

1. Processes and threads

A process is a program in execution. It has its own address space, open files and a process control block (PCB) that stores its state, program counter, registers and scheduling information. A process moves between the states new, ready, running, waiting and terminated. A thread is a smaller unit of execution inside a process. Threads of the same process share code, data and open files, but each thread has its own stack and registers. Creating a thread is cheaper than creating a process, and switching between threads is faster than a full context switch between processes. A context switch saves the state of the running process in its PCB and loads the state of the next one; during the switch the CPU does no useful work, so too many switches waste time.

2. CPU scheduling

The short-term scheduler picks which ready process gets the CPU next. First-Come First-Served (FCFS) runs processes in arrival order; it is simple but short jobs can wait behind a long one, which is called the convoy effect. Shortest Job First (SJF) gives the minimum average waiting time, but the length of the next CPU burst must be predicted. Priority scheduling runs the most important process first; low-priority processes may starve, and aging solves this by slowly raising the priority of processes that wait a long time. Round Robin (RR) gives every process a fixed time quantum, for example 10 milliseconds, and then moves it to the back of the ready queue. A very large quantum makes RR behave like FCFS; a very small quantum causes too many context switches. Useful measures are turnaround time, waiting time and response time.

3. Deadlocks

A deadlock is a situation where a set of processes are blocked forever because each one holds a resource and waits for a resource held by another process in the set. Four conditions must hold at the same time: mutual exclusion, hold and wait, no preemption, and circular wait. Deadlock prevention breaks one of these conditions, for example by numbering resources and always requesting them in increasing order, which makes circular wait impossible. Deadlock avoidance checks every request in advance: the Banker's algorithm grants a request only if the system stays in a safe state. Detection and recovery allows deadlocks, finds them with a wait-for graph, and then recovers by terminating a process or taking resources away from it.

4. Memory management and paging

Paging divides physical memory into fixed-size blocks called frames and divides each process's logical memory into blocks of the same size called pages. A page table maps every page number to a frame number. A logical address is split into a page number and an offset; the page number is looked up in the page table and the offset is added to the start of the frame. Paging removes external fragmentation because any free frame can hold any page, but the last page of a process may be partly empty, which is internal fragmentation. Looking up the page table on every memory access is slow, so a small fast cache called the Translation Lookaside Buffer (TLB) stores recent page-to-frame translations. Segmentation is a different scheme that divides memory into variable-size segments such as code, stack and heap.

5. Virtual memory and page replacement

Virtual memory lets a process run even when only part of it is in physical memory. Pages are loaded only when they are needed, which is called demand paging. When a process touches a page that is not in memory, a page fault occurs: the operating system finds the page on disk, loads

it into a free frame and restarts the instruction. If no frame is free, a page replacement algorithm chooses a victim page. FIFO replaces the oldest page and can suffer from Belady's anomaly, where more frames cause more page faults. The Optimal algorithm replaces the page that will not be used for the longest time; it gives the fewest faults but needs knowledge of the future. Least Recently Used (LRU) replaces the page that has not been used for the longest time and is a good practical approximation. When processes spend more time swapping pages than executing, the system is thrashing.

6. File systems and disk scheduling

A file system organises data on disk into files and directories. Contiguous allocation stores a file in consecutive blocks, which is fast to read but causes external fragmentation. Linked allocation stores a pointer to the next block inside each block, so there is no fragmentation but random access is slow. Indexed allocation keeps all block addresses of a file in an index block, as Unix inodes do. Disk scheduling decides the order in which pending disk requests are served to reduce head movement. FCFS serves requests in arrival order. Shortest Seek Time First (SSTF) serves the nearest request first but may starve far requests. SCAN, also called the elevator algorithm, moves the head from one end of the disk to the other and serves requests on the way, then reverses direction.